ЛР 4. Интеграция, формы, отладка и автотесты

Дисциплина:«Протоколы и интерфейсы информационных систем».

> 💡 Что вы сделаете в лабораторной работе
>
> **ЧТО ВЫ СДЕЛАЕТЕ В ЭТОЙ ЛАБОРАТОРНОЙ**
>
> • подключите Angular HttpClient к Flask API через dev proxy;
>
> • реализуйте HTTP-сервис и сохранение reactive form;
>
> • пройдете полный CRUD-сценарий через браузер;
>
> • проверите обработку ошибок и выполнить отладку frontend/backend;
>
> • запустите unit/integration и E2E-проверки и обновить CI.

1. Перед практикой минимум теории

HttpClient возвращает *Observable*: запрос отправляется при подписке. Компонент отображает состояния загрузки, результата и ошибки. Тип *Ticket* помогает проверять код TypeScript, но сам по себе не проверяет произвольный JSON во время выполнения.

```
this.api.get(1).subscribe({
  next: ticket => console.log(ticket.title),
  error: error => console.error(error),
});
```

Проверка формы в браузере помогает пользователю, серверная проверка защищает контракт API. Обе нужны: запрос можно отправить напрямую. Пока сохранение выполняется, повторное нажатие не должно создавать второй запрос.

Браузер различает *origin* по схеме, хосту и порту. *Dev proxy* отправляет */api* с сервера *Angular* на *Flask*; для браузера обращение остаётся к origin фронтенда. Это настройка разработки. На развёрнутом сервере аналогичную маршрутизацию выполняет *reverse proxy*. Не включайте разрешение любых *origin*, чтобы скрыть ошибку адреса запроса.

Модульный тест сервиса проверяет *HTTP-контракт* без настоящего сервера. Интеграционный тест Flask проверяет MySQL. *E2E* запускает настоящий браузер и проверяет пользовательский сценарий. Эти проверки дополняют друг друга.

2. Подготовка проекта

Из корня, с активированным Python-окружением:

```
git status
python tools/prepare_lr4.py --apply
cd frontend
npm ci
cd ..
```

Перед подготовкой сохраните свои изменения *commit-ом.* Скрипт переносит прежние файлы frontend в *.course-backups* и устанавливает учебный комплект. Не выполняйте его повторно после начала реализации: это опять заменит frontend. Если запустили случайно, восстановите свои файлы из свежей резервной копии или Git. Каталог .course-backups не публикуется.

Откройте *frontend/src/app*: в *pages* — страницы, в *services* — HTTP-сервис, в *models* — модели.
В маршрутах уже предусмотрены список, создание, карточка, редактирование и оборудование.
До выполнения следующих шагов ошибки TODO и падающие приёмочные тесты ожидаемы.

3. Практика подключение API и формы

3.1. Зарегистрируйте HttpClient

Замените *frontend/src/app/app.config.ts* следующим содержимым:

```
# frontend/src/app/app.config.ts
mport { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';
export const appConfig: ApplicationConfig = {
  providers: [provideBrowserGlobalErrorListeners(), provideRouter(routes),
provideHttpClient()],
};
```

Без *provider* Angular не сможет внедрить *HttpClient* в сервис. Найдите место вызова *inject* в следующем шаге и объясните связь с *provider.*

3.2. Настройте proxy

Заполните *frontend/proxy.conf.json*:

```
# frontend/proxy.conf.json
{ "/api/**": { "target": "http://127.0.0.1:5000", "secure": false } }
```

Проверьте *frontend/package.json*: команда *start* должна подключать этот конфигурационный файл. Flask работает на порту 5000, Angular — на 4200. После изменения proxy перезапускайте Angular. URL в сервисе начинается с */api*, а не с абсолютного адреса Flask.

3.3. Реализуйте HTTP-сервис

Откройте *frontend/src/app/services/tickets-api.ts*. Замените заглушки полным определением ниже.

Вводите методы по одному: сначала *list/get*, затем *create/update/delete*, затем справочники. После каждого метода сопоставьте его с маршрутом Flask из ЛР №2.

```
# frontend/src/app/services/tickets-api.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpErrorResponse, HttpParams } from '@angular/common/http';
import { Equipment, Stats, Ticket, TicketEvent, TicketInput, TicketStatus } from
'../models/ticket';
@Injectable({ providedIn: 'root' })
export class TicketsApi {
  private readonly http = inject(HttpClient);
  list(q = '', mode = 'all') {
    let params = new HttpParams().set('q', q.trim());
    if (mode === 'open') params = params.set('scope', 'open');
    if (mode === 'new' || mode === 'in_progress' || mode === 'closed')
      params = params.set('status', mode);
    return this.http.get<Ticket[]>('/api/tickets', { params });
  }
  get(id: number) {
    return this.http.get<Ticket>(`/api/tickets/${id}`);
  }
  create(input: TicketInput) {
    return this.http.post<Ticket>('/api/tickets', input);
  }
  update(id: number, input: TicketInput & { status: TicketStatus }) {
    return this.http.patch<Ticket>(`/api/tickets/${id}`, input);
  }
  delete(id: number) {
    return this.http.delete<void>(`/api/tickets/${id}`);
  }
  equipment() {
    return this.http.get<Equipment[]>('/api/equipment');
  }
  stats() {
    return this.http.get<Stats>('/api/stats');
  }
  events(id: number) {
    return this.http.get<TicketEvent[]>(`/api/tickets/${id}/events`);
  }
```

```
}
export function errorMessage(error: unknown): string {
  if (error instanceof HttpErrorResponse) {
    if (error.status === 0)
      return 'Нет связи с сервером. Проверьте подключение и повторите попытку.';
    if (typeof error.error?.error?.message === 'string') return error.error.error.message;
  }
  return 'Не удалось выполнить запрос. Повторите попытку.';
}
```

HttpParams неизменяем: *set* возвращает новый объект. Поэтому результат присваивается обратно. Режим openпередаёт *scope=open*, отдельный статус — *status*. *DELETE* возвращает пустое тело 204. В обработчике ошибки *status=0* означает сетевой сбой; структурированную ошибку сервера берём из *error.message* внутри JSON-ответа. Не выводите пользователю трассировку Python.

Промежуточная проверка из *frontend*:

```
npm test -- --watch=false
```

Тесты сервиса теперь должны пройти; тесты сохранения формы могут оставаться красными до шага 5.
При ошибке «не найден запрос» проверьте подписку, HTTP-метод и адрес.

3.4. Подключите сохранение формы

В *frontend/src/app/pages/ticket-form/ticket-form.ts* прочитайте объявление *form* и метод *load*. Загрузка использует *forkJoin*: ждёт справочник оборудования и, при редактировании, заявку.
Для новой заявки вместо запроса существующей используется of(null).

Найдите *save()* и замените только этот метод. Фрагмент вставляется внутри класса *TicketForm*:

```
# frontend/src/app/pages/ticket-form/ticket-form.ts
save() {
    if (this.saving()) return;
    const value = this.form.getRawValue();
    this.form.controls.title.setValue(value.title.trim());
    this.form.markAllAsTouched();
    if (this.form.invalid) return;
    const input = {
      title: value.title.trim(),
      description: value.description.trim(),
      equipmentId: Number(value.equipmentId),
    };
    this.saving.set(true);
```

```
    this.error.set('');
    const request = this.id
      ? this.api.update(this.id, { ...input, status: value.status })
      : this.api.create(input);
    request.subscribe({
      next: (ticket) => {
        this.saving.set(false);
        this.router.navigate(['/tickets', ticket.id]);
      },
      error: (err) => {
        this.error.set(errorMessage(err));
        this.saving.set(false);
      },
    });
  }
```

Проверьте порядок: убрать пробелы → показать ошибки → остановиться при невалидной форме → заблокировать повторное сохранение → отправить запрос → снять блокировку при любом исходе. При ошибке введённые значения остаются в форме. При успехе открывается карточка созданной записи.

В HTML уже подготовлены сообщения проверки и disabled для кнопки: найдите эти привязки.

4. Запуск и пользовательский сценарий

4.1. Запустите весь проект

Первый терминал, активированное окружение, каталог *backend*:

```
python -m flask --app app run --debug --port 5000
```

В backend/.env должны остаться STORAGE=mysql и параметры из ЛР №3. Второй терминал, frontend:

```
# backend/.env
npm start
```

Откройте *http://localhost:4200. В DevTools* → *Network* найдите */api/tickets:* ожидается 200 и JSON, а не HTML. При 502 проверьте Flask и адрес proxy; при 501 - незавершённый метод сервера; при 503 - соединение MySQL. Если список пуст, проверьте seed и выбранные фильтры.

4.2. Пройдите пользовательский сценарий

1. Откройте список: проверьте начальные заявки, переключите статусы, введите часть названия.

2. Создайте заявку «Настроить рабочее место», выберите оборудование, добавьте описание.
3. Убедитесь, что открылась карточка; запишите ID, HTTP-метод, статус и тело запроса из Network.
4. Измените описание и статус на «В работе». Проверьте PATCH и новое событие в истории.
5. Перезагрузите браузер и Flask. Запись должна сохраниться в MySQL.
6. Сверьте сводку с количеством заявок, откройте справочник оборудования.
7. Удалите только свою тестовую заявку. Проверьте DELETE 204 и отсутствие записи в списке.

Для каждого изменения определите таблицы, которых оно касается. Приложите в отчёт одну цепочку: действие → запрос JSON → маршрут → SQL → ответ → обновление интерфейса.

5. Надёжность и отладка

1. В форме введите два символа в название: отправка должна блокироваться.
2. Введите одни пробелы: после нормализации форма также не отправляется.
3. Остановите Flask и попробуйте открыть список: появится понятное сообщение о связи.
4. Запустите Flask снова и воспользуйтесь повторной загрузкой.
5. Откройте карточку несуществующего ID: сервер вернёт 404, интерфейс покажет ошибку.
6. Поставьте breakpoint в save() через Sources браузера (исходники TypeScriptдоступны в dev).
   > Затем breakpoint в обработчике Flask через конфигурацию VS Code из ЛР №2. Не запускайте два Flask на одном порту: остановите терминальный экземпляр перед debugger.
7. Пройдите одно сохранение и сравните equipmentId в форме, JSON и аргументах репозитория.

Дополнительное наблюдение: прямой GET из консоли страницы на h*ttp://127.0.0.1:5000/api/health* может блокироваться политикой *origin*, тогда как /*api/health* проходит через proxy. CORS проверяет браузер; успешный запрос из Python не подтверждает настройку CORS.

6. Проверки

6.1. Запустите модульные и интеграционные проверки

Из frontend:

```
npm test -- --watch=false
npm run build
```

Ожидаются 9 подготовленных *frontend-тестов* и успешная сборка. Прочитайте тесты в *services/tickets-api.spec.ts, pages/tickets/tickets.spec.ts*, *pages/ticket-form/ticket-form.spec.ts*: они проверяют фильтры, HTTP-методы, отображение и сохранение формы.

6.2. Запустите браузерные тесты

MySQL, Flask и Angular должны быть запущены. *E2E* работает через UI и создаёт/удаляет собственные тестовые заявки. Используйте локальную учебную базу. Из корня в третьем терминале:

```
cd e2e
npm ci
npx playwright install chromium
npm test
```

Ожидаются 3 успешных браузерных сценария. На Linux Playwright может запросить системные библиотеки; установите зависимости браузера по официальной инструкции для своей ОС. При сбое откройте отчёт командой *npx playwright show-report*, сверьте URL в *playwright.config.ts*.

7. Самостоятельная часть и обновление CI

Выберите одно расширение тестов: ошибка POSTсохраняет введённые значения; повторное нажатие во время *saving* не создаёт второй запрос; ошибка DELETEоставляет карточку доступной.

Сначала сформулируйте ожидаемое поведение словами, затем воспроизведите ошибку и добавьте тест.

В *.github/workflows* проверьте фронтенд-команду: после замены страниц она должна запускать *npm test -- --watch=false и npm run build*, вместо исходного *npm run check:starter*, которого больше нет в новом *package.json*.

Замените строку *run на npm ci&& npm test -- - watch=false&& npm run build*.
Backend-тесты MySQLтребуют отдельного сервиса БД в CI. Локальные результаты приложите в отчёт; не называйте имеющуюся frontend-проверку полной проверкой всей системы.

8. Требования к завершённому проекту

Обязательны CRUD через браузер, поиск и фильтры, реляционное хранение, история изменений, валидация с обеих сторон, обработка сетевых ошибок, воспроизводимый запуск и тесты.
В README своего репозитория укажите установку, настройку .env, старт двух серверов и проверки. Не публикуйте .env, пароли, зависимости и резервные копии. В работе нет авторизации и ролей: это учебная локальная ИС, а не готовая служба для публичного доступа.

9. Что сдавать

> 💡 Перечень
>
> • актуальный проект it-service-desk с выполненной лабораторной;
>
> • результаты предусмотренных в методичке проверок/тестов;

> • демонстрацию ожидаемого пользовательского или серверного сценария;
>
> • ответы на вопросы и результаты.

10. Материалы для подготовки

1. Angular HTTP: запросы, обработка ошибок и тестирование.
2. Angular Reactive Forms.
3. Angular dev server proxy.
4. Angular testing.
5. MDN CORS.
6. Playwright introduction.
7. Flask debugging.